PYTHON SOURCE CODE

L.D. COLLEGE OF ENGINEERING, AHMEDABAD



SMART TRAFFIC SIGNAL CONTROLLER

Subject: Fuzzy logic & Genetic Programming

Submitted By:

Name: Patel Dip N. (240280106096)

Patel Vraj M.(240280106113)

Patel Aastha M.(240280106092)

Semester: 5

Department : Computer Engineering

Academic Year: 2026–2027

```
from dataclasses import dataclass
from typing import Dict, List, Tuple


def trimf(x: float, abc: Tuple[float, float, float]) -> float:
    a, b, c = abc

    if a == b == c:
        return 1.0 if x == a else 0.0
    if x == b:
        return 1.0
    if x <= a or x >= c:
        return 0.0
    if x < b:
        return (x - a) / (b - a) if b != a else 1.0
    return (c - x) / (c - b) if c != b else 1.0


def trapmf(x: float, abcd: Tuple[float, float, float, float]) ->
float:
    a, b, c, d = abcd

    if b <= x <= c:
        return 1.0
    if x <= a or x >= d:
        return 0.0
    if a < x < b:
        return (x - a) / (b - a) if b != a else 1.0
    if c < x < d:
        return (d - x) / (d - c) if d != c else 1.0
    return 0.0


DENSITY_MF = {
    "low": lambda x: trapmf(x, (0, 0, 8, 16)),
    "medium": lambda x: trimf(x, (10, 20, 30)),
    "high": lambda x: trapmf(x, (24, 32, 40, 40)),
}

OUTPUT_MF = {
    "short": lambda x: trapmf(x, (10, 10, 18, 28)),
    "medium": lambda x: trimf(x, (20, 35, 48)),
    "long": lambda x: trapmf(x, (40, 50, 60, 60)),
}


def waiting_memberships(wait_seconds: float) -> Dict[str, float]:
    w = max(0.0, min(180.0, float(wait_seconds)))

    return {
        "short": trapmf(w, (0, 0, 20, 50)),
        "medium": trimf(w, (30, 75, 120)),
        "long": trapmf(w, (90, 130, 180, 180)),
    }


@dataclass
class ControllerResult:
```

```
    density: float
    waiting_time: float
    green_time: float
    density_memberships: Dict[str, float]
    waiting_memberships: Dict[str, float]
    rule_strengths: Dict[str, float]


def fuzzy_green_time(
    traffic_density: float, waiting_time: float
) -> ControllerResult:

    density = max(0.0, min(40.0, float(traffic_density)))
    wait = max(0.0, min(180.0, float(waiting_time)))

    dm = {name: mf(density) for name, mf in DENSITY_MF.items()}
    wm = waiting_memberships(wait)

    rules = {
        "low & short -> short": min(dm["low"], wm["short"]),
        "low & medium -> short": min(dm["low"], wm["medium"]),
        "low & long -> medium": min(dm["low"], wm["long"]),
        "medium & short -> medium": min(dm["medium"], wm["short"]),
        "medium & medium -> medium": min(dm["medium"], wm["medium"]),
        "medium & long -> long": min(dm["medium"], wm["long"]),
        "high & short -> long": min(dm["high"], wm["short"]),
        "high & medium -> long": min(dm["high"], wm["medium"]),
        "high & long -> long": min(dm["high"], wm["long"]),
    }

    xs = [10.0 + i * 0.25 for i in range(201)]
    aggregated: List[float] = []

    short_strength = max(
        rules["low & short -> short"],
        rules["low & medium -> short"],
    )

    medium_strength = max(
        rules["low & long -> medium"],
        rules["medium & short -> medium"],
        rules["medium & medium -> medium"],
    )

    long_strength = max(
        rules["medium & long -> long"],
        rules["high & short -> long"],
        rules["high & medium -> long"],
        rules["high & long -> long"],
    )

    for x in xs:
        membership = max(
            min(short_strength, OUTPUT_MF["short"](x)),
            min(medium_strength, OUTPUT_MF["medium"](x)),
            min(long_strength, OUTPUT_MF["long"](x)),
        )
        aggregated.append(membership)

    area = sum(aggregated)
```

```
    if area == 0:
        green = 30.0
    else:
        green = sum(
            x * mu for x, mu in zip(xs, aggregated)
        ) / area

    green = max(10.0, min(60.0, green))

    return ControllerResult(
        density=density,
        waiting_time=wait,
        green_time=green,
        density_memberships=dm,
        waiting_memberships=wm,
        rule_strengths=rules,
    )


def recommend_signal(density: float, wait: float) -> None:
    result = fuzzy_green_time(density, wait)

    print("\nSMART TRAFFIC SIGNAL CONTROLLER")
    print("-" * 36)
    print(f"Traffic density : {result.density:.1f} vehicles")
    print(f"Waiting time    : {result.waiting_time:.1f} seconds")

    print("Density membership:", {
        key: round(value, 3)
        for key, value in result.density_memberships.items()
    })

    print("Waiting membership:", {
        key: round(value, 3)
        for key, value in result.waiting_memberships.items()
    })

    print(f"Recommended green time: {result.green_time:.1f} seconds")
    print("\nNote: Yellow/all-red intervals and safe signal
sequencing")
    print("must be handled by a separate safety-critical controller.")


def run_examples() -> None:
    examples = [
        ("Low traffic", 5, 15),
        ("Medium traffic", 20, 60),
        ("High traffic", 35, 100),
    ]

    print("ILLUSTRATIVE TEST CASES")
    print("=" * 64)

print(f"{'Case':<16}{'Density':>10}{'Wait(s)':>12}{'Green(s)':>14}")
    print("-" * 64)

    for name, density, wait in examples:
        result = fuzzy_green_time(density, wait)
        print(
            f"{name:<16}{density:>10.1f}"
            f"{wait:>12.1f}{result.green_time:>14.1f}"
```

```
        )


if __name__ == "__main__":
    run_examples()

    try:
        answer = input(
            "\nEnter traffic density (0-40 vehicles), "
            "or press Enter to exit: "
        ).strip()

        if answer:
            density_value = float(answer)
            wait_value = float(
                input("Enter waiting time (0-180 seconds): ").strip()
            )
            recommend_signal(density_value, wait_value)

    except ValueError:
        print("Invalid input. Please enter numeric values.")
```